

S4-F — Headless TORCS w WSL

Contents

TORCS headless w WSL — równoległe workery Optuny na teacher_wp	1
Etap A — uruchom JEDNĄ instancję headless	1
Etap B — skalowanie do N workerów (po zadziałaniu A)	3
Uwagi	3

TORCS headless w WSL — równoległe workery Optuny na teacher_wp

English version: [README_WSL.md](#)

Cel: uruchomić N instancji TORCS 1.3.7 w trybie headless w WSL Ubuntu, gdzie każda jest workerem Optuny dołączonym do **tego samego** chmurowego study Postgresa teacher_wp, aby szybciej przeszukiwać parametry teachera v3 (cel: okrążenie ~80 s).

TORCS budujemy z **tego samego źródła fmirus**, którego używa Dockerfile organizatorów, więc fizyka jest identyczna jak w kontenerze konkursowym. Pomijamy Docker/VS Code/Ollama — praca bezpośrednio w WSL jest lżejsza i daje bezpośredni dostęp do kodu S4-F na /mnt/d.

Tryb wyjściu to display mode = results only (zob. practice.xml), więc TORCS działa **bez renderowania 3D**, przez co jest ograniczony przez CPU, a nie przez czas rzeczywisty. Może działać **szybciej** niż w czasie rzeczywistym i skaluje się z liczbą rdzeni.

Etap A — uruchom JEDNĄ instancję headless

1. Aktualizacja apt (w Ubuntu)

```
sudo apt-get update
```

2. Instalacja zależności do budowy + headless

```
sudo apt-get install -y --no-install-recommends \
build-essential git wget unzip \
libgl1-mesa-dev libglu1-mesa-dev freeglut3-dev \
libpng-dev libjpeg-dev libopenal-dev libalut-dev libvorbis-dev libogg-dev \
libxi-dev libxmu-dev libxrender-dev libxrandr-dev libxxf86vm-dev libplib-dev \
xvfb xautomation python3 python3-pip python3-venv
```

3. Budowa i instalacja TORCS 1.3.7 (dokładny przepis organizatorów)

```
cd ~
git clone --depth=1 https://github.com/fmirus/torcs-1.3.7.git torcs-src
```

```
cd torcs-src
./configure --prefix=/usr/local/torcs
make # ~20-40 min. NIE dodawaj -j - Makefile TORCS-a ma wyścigi.
sudo make install
sudo make datainstall
```

Dodaj TORCS do PATH:

```
echo 'export PATH=/usr/local/torcs/bin:$PATH' >> ~/.bashrc
export PATH=/usr/local/torcs/bin:$PATH
```

4. Instalacja auta/sterownika serwera SCR (z naszego zipa)

```
cp "/mnt/d/IBM_competition/SAC/S3_B/S4-F/TORCS_SETUP/Setup Guide/build your own container/torcs-con
sudo mkdir -p /usr/local/share/games/torcs/drivers
sudo unzip -o ~/scr_server.zip -d /usr/local/share/games/torcs/drivers/
sudo rm -rf /usr/local/share/games/torcs/drivers/__MACOSX
```

5. Kontrola poprawności (tor + sterownik obecne, binarka na PATH)

```
ls /usr/local/share/games/torcs/tracks/road/corkscrew # tor Laguna Seca
ls /usr/local/share/games/torcs/drivers/scr_server # sterownik SCR
which torcs # /usr/local/torcs/bin/torcs
```

6. Środowisko Pythona dla sterownika

```
python3 -m venv ~/torcs-venv
source ~/torcs-venv/bin/activate
pip install --upgrade pip
pip install optuna psycogp2-binary numpy
pip install torch --index-url https://download.pytorch.org/whl/cpu # config.py importuje torch (wh
echo 'source ~/torcs-venv/bin/activate' >> ~/.bashrc
```

7. Wskaż sterownikowi chmurowe study Postgresa

Czyta connection string z istniejącego pliku .pg_url (hasła nigdy nie wpisujesz ręcznie):

```
echo 'export OPTUNA_STORAGE="$(cat "/mnt/d/IBM_competition/SAC/S3_B/S4-F/.pg_url")"' >> ~/.bashrc
source ~/.bashrc
echo "$OPTUNA_STORAGE" | sed -E 's#://[^@]+@#://***:***@#' # powinno wypisać zamaskowany URL Postgr
```

8. Uruchom JEDNEGO workera

```
cd "/mnt/d/IBM_competition/SAC/S3_B/S4-F"
python3 optuna_teacher_linux.py --study-name teacher_wp --n-laps 1 --port 3001
```

Oczekiwane: Client connected on 3001, a następnie strumień [trial N] <lap>s [...vs DAgger] | done=k | <RATE>/hr | best=.... **Zanotuj tempo /hr** — to ono decyduje, ile równoległych instancji ma sens.

Jeśli zawiesza się na Waiting for server on 3001... torcs -r nie dogadał się ze scr_server na Twojej kompilacji. Opcje, po kolei:

1. Daj wszystkiemu wirtualny wyświetlacz (niektóre kompilacje go wymagają nawet dla results-only):

```
xvfb-run -a python3 optuna_teacher_linux.py --study-name teacher_wp --n-laps 1 --port 3001
```

2. Użyj sprawdzonej ścieżki menu z gym_torcs (Xvfb + naciśnięcia klawiszy xte):

```
python3 optuna_teacher_linux.py --study-name teacher_wp --n-laps 1 --port 3001 --launch menu --display
```

3. Dalej nie działa -> daj znać. Podglądamy menu przez VNC (x11vnc -display :1 -nopw &, potem przeglądarka VNC na localhost:5900) i poprawiamy sekwencję klawiszy.

Etap B — skalowanie do N workerów (po zadziałaniu A)

Każdy worker to własny torcs + własny port SCR + własne ~/.torcs (żeby konfiguracje się nie kolidowały). Zgrubny przepis na kolejnego workera (pokazany #2):

```
export HOME_ALT=~/.torcs_w2
mkdir -p "$HOME_ALT"
HOME="$HOME_ALT" python3 optuna_teacher_linux.py --study-name teacher_wp --n-laps 1 --port 3002
```

(Sterownik zapisuje practice.xml w \$HOME/.torcs i podmienia port scr_server, więc nadpisanie HOME izoluje każdą instancję.) **Nie przesadzaj z liczbą** — każdy torcs zjada ~1 rdzeń; zostaw zapas. Dokładne komendy ustalimy, gdy zobaczymy tempo /hr z A i liczbę Twoich rdzeni.

Uwagi

- Ewaluacja (teacher przez UDP snakeoil) jest identyczna jak na Windowsie; różni się tylko sposób uruchomienia TORCS-a. To samo study Optuny, więc optuna_teacher_v3.py --mode report / --mode export-best na Windowsie nadal odczytuje wszystko, co dopisują te workery.
- Natywny sweep na Windowsie (optuna_teacher_single.py) może działać równolegle na tym samym study — Postgres jest wspólnym koordynatorem.
- torcs -r uruchamia wyścig i kończy działanie, więc sterownik startuje go ponownie przy każdym trialu (tanie na Linuksie, bez menu). To także sprawia, że każdy trial jest deterministyczny.